

Week 3: Final Report

MLFQ Scheduler for xv6-RISC-V

Priority Boosting, Starvation Prevention, and Complete Validation

Abdullah Khan Sherwani 27880

Muhammad Anas Tabbal 28998

3rd December 2025

Contents

1	Project Overview	3
1.1	Introduction	3
1.2	Project Timeline	3
2	Final MLFQ Configuration	3
2.1	Queue Structure	3
2.2	Scheduling Parameters	3
3	Priority Boosting Implementation	4
3.1	Starvation Problem	4
3.2	Solution: Periodic Boosting	4
3.3	Boost Function	4
3.4	boostproc System Call	5
4	Extended Boost Test	5
4.1	boostlong.c Test Program	5
5	Complete System Architecture	7
5.1	Data Flow	7
5.2	MLFQ State Machine	7
5.3	Files Modified	8
6	Test Results	8
6.1	Priority Boost Test (Long)	8
6.2	Priority Boost Test (Short)	9
6.3	Starvation Prevention Test	10
6.4	Priority Boost Test (Long)	11
6.5	Starvation Prevention Test	12
6.6	Test Summary	13
6.7	Key Observations	13

7	Performance Characteristics	13
7.1	Responsiveness	13
7.2	Throughput	13
7.3	Fairness	14
8	Conclusion	14
8.1	Achievements	14
8.2	MLFQ Rules Implemented	14
8.3	Future Improvements	14
9	Appendix: System Call Reference	15
9.1	getprocinfo	15
9.2	boostproc	15

1 Project Overview

1.1 Introduction

This final report documents the complete implementation of a Multi-Level Feedback Queue (MLFQ) scheduler for xv6-RISC-V. The project replaces xv6's default round-robin scheduler with a sophisticated priority-based scheduler that:

- Automatically adapts to process behavior
- Favors interactive and I/O-bound processes
- Prevents starvation through periodic priority boosting
- Maintains fairness across all process types

1.2 Project Timeline

Week	Focus	Deliverables
1	Design & Foundation	Data structures, syscalls, design document
2	Core Implementation	Scheduler logic, demotion, test programs
3	Boosting & Validation	Priority boost, starvation prevention, final tests

Table 1: Project Timeline

2 Final MLFQ Configuration

2.1 Queue Structure

Queue	Priority	Time Quantum	Process Type
Q0	Highest	2 ticks (200ms)	Interactive, I/O-bound
Q1	High	4 ticks (400ms)	Mixed workloads
Q2	Medium	8 ticks (800ms)	Compute-light
Q3	Lowest	16 ticks (1.6s)	CPU-bound, batch

Table 2: Final Queue Configuration

2.2 Scheduling Parameters

```

1 #define NMLFQ 4           // Number of priority queues
2 #define BOOST_INTERVAL 50 // Boost all processes every 50
   ticks
3
4 int mlfq_time_quanta[NMLFQ] = {2, 4, 8, 16}; // Ticks per queue
5 uint64 last_boost_time = 0; // Track time since last boost

```

Listing 1: MLFQ Parameters (kernel/proc.c)

3 Priority Boosting Implementation

3.1 Starvation Problem

Without priority boosting, a CPU-bound process demoted to Q3 could starve if:

- Higher-priority processes continuously arrive
- The scheduler never reaches Q3
- The process waits indefinitely

3.2 Solution: Periodic Boosting

Every 50 ticks (5 seconds), all processes are boosted back to Q0:

```
1 void
2 scheduler(void)
3 {
4     struct proc *p;
5     struct cpu *c = mycpu();
6
7     c->proc = 0;
8     for(;;){
9         intr_on();
10        intr_off();
11
12        // Check if it's time for priority boosting
13        acquire(&tickslock);
14        uint64 current_ticks = ticks;
15        if(current_ticks - last_boost_time >= BOOST_INTERVAL) {
16            last_boost_time = current_ticks;
17            release(&tickslock);
18            boost_all_priorities(); // BOOST ALL PROCESSES TO Q0
19        } else {
20            release(&tickslock);
21        }
22
23        // ... MLFQ scheduling logic ...
24    }
25 }
```

Listing 2: Priority Boosting in Scheduler (kernel/proc.c)

3.3 Boost Function

```
1 void
2 boost_all_priorities(void)
3 {
4     struct proc *p;
5     int boosted_count = 0;
```

```

6
7  for(p = proc; p < &proc[NPROC]; p++) {
8      acquire(&p->lock);
9      if(p->state == RUNNABLE || p->state == RUNNING || p->state ==
10         SLEEPING) {
11          if(p->priority != 0 || p->time_slices != 0) {
12              boosted_count++;
13          }
14          p->priority = 0;           // Reset to highest priority
15          p->time_slices = 0;       // Reset time slice counter
16      }
17      release(&p->lock);
18  }

```

Listing 3: boost_all_priorities Function (kernel/proc.c)

3.4 boostproc System Call

Added a manual boost syscall for testing:

```

1  uint64
2  sys_boostproc(void)
3  {
4      extern void boost_all_priorities(void);
5
6      boost_all_priorities();
7
8      return 0;
9  }

```

Listing 4: boostproc System Call (kernel/sysproc.c)

4 Extended Boost Test

4.1 boostlong.c Test Program

A comprehensive test that demonstrates the complete MLFQ cycle with tabular output. The test tracks three child processes through multiple boost cycles, showing demotions and automatic priority boosts.

```

1  #include "kernel/types.h"
2  #include "kernel/stat.h"
3  #include "user/user.h"
4
5  void cpu_work(void) {
6      volatile int x = 0;
7      for(int i = 0; i < 1000000; i++) x++;
8  }
9
10 int main(int argc, char *argv[]) {

```

```

11  printf("=== MLFQ Priority Boost Demonstration ===\n");
12  printf("Tracking 3 processes through boost cycles\n");
13  printf("Boost interval: 50 ticks\n\n");
14
15  int pids[3];
16  for(int i = 0; i < 3; i++) {
17      pids[i] = fork();
18      if(pids[i] == 0) {
19          // Child process - CPU-bound work
20          struct procinfo info;
21          int last_q = -1, last_slices = -1;
22          int boost_count = 0;
23          int started_tracking = 0;
24
25          for(int iter = 0; iter < 500; iter++) {
26              cpu_work();
27              if(getprocinfo(&info) == 0) {
28                  // Wait for first boost before tracking
29                  if(!started_tracking && info.priority == 0 &&
30                     info.time_slices == 0 && iter > 10) {
31                      started_tracking = 1;
32                      printf("TICK | QUEUE | SLICES | EVENT\n");
33                      printf("-----|-----|-----|-----\n");
34                  }
35
36                  if(started_tracking) {
37                      int tick = uptime();
38                      // Detect boost (priority decreased to 0)
39                      if(last_q > 0 && info.priority == 0) {
40                          boost_count++;
41                          printf(" %d |  Q%d  |   %d   | BOOST #%d\n",
42                             tick, info.priority, info.time_slices,
43                             boost_count);
44                      }
45                      // Detect demotion
46                      else if(info.priority > last_q && last_q >= 0) {
47                          printf(" %d |  Q%d  |   %d   | Demoted from Q%d\n",
48                             tick, info.priority, info.time_slices,
49                             last_q);
50                      }
51                      last_q = info.priority;
52                      last_slices = info.time_slices;
53                  }
54                  if(boost_count >= 3) break;
55              }
56              printf("\n--- Process complete: %d boosts detected ---\n",
57                 boost_count);
58              exit(0);
59          }
60      }
61  }

```

```

59 }
60
61 // Parent waits
62 for(int i = 0; i < 3; i++) wait(0);
63 printf("\n=== All processes completed ===\n");
64 exit(0);
65 }

```

Listing 5: Extended Boost Test (user/boostlong.c)

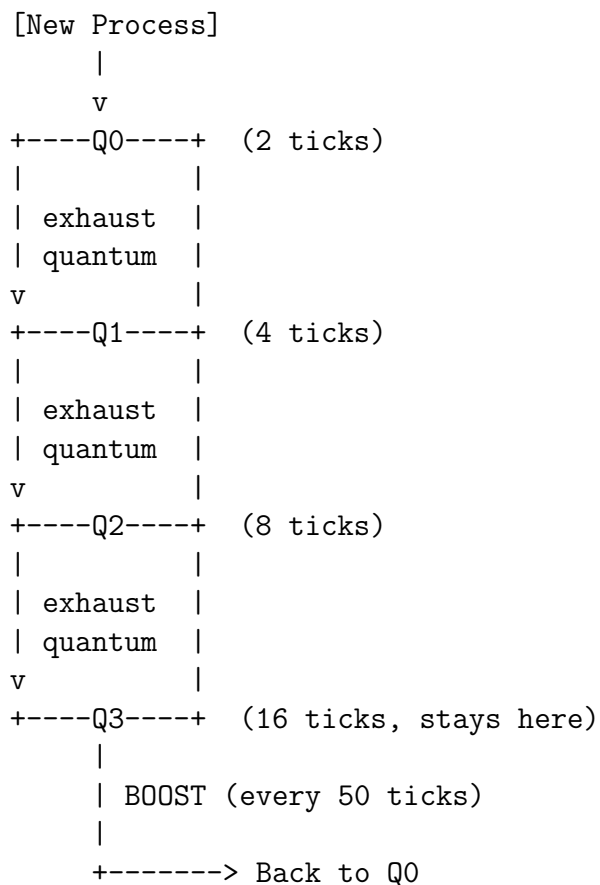
5 Complete System Architecture

5.1 Data Flow

1. **Process Creation:** New process starts at Q0 with `time_slices=0`
2. **Timer Interrupt:** On each tick, `time_slices++`
3. **Demotion Check:** If `time_slices >= quantum[priority]`, demote
4. **Scheduler:** Always picks from highest non-empty queue
5. **Boost Check:** Every 50 ticks, all processes $\rightarrow$ Q0

5.2 MLFQ State Machine

A process transitions through the following states:



5.3 Files Modified

File	Modification
kernel/proc.h	Added MLFQ fields to struct proc
kernel/proc.c	MLFQ scheduler, boost function, initialization
kernel/trap.c	Time slice tracking, demotion logic
kernel/sysproc.c	getprocinfo, boostproc syscalls
kernel/syscall.h	Syscall numbers
kernel/syscall.c	Syscall registration
user/user.h	User-space declarations
user/usys.pl	Syscall stubs
user/*.c	Test programs
Makefile	Build configuration

Table 3: Complete File List

6 Test Results

6.1 Priority Boost Test (Long)

The `boostlong` test demonstrates automatic priority boosting every 50 ticks, showing the complete demotion cycle ($Q0 \rightarrow Q1 \rightarrow Q2 \rightarrow Q3$) and how processes at $Q3$ stay at $Q3$ until boosted back to $Q0$.

```

$
$ boostlong

=====
MLFQ BOOST TEST - SINGLE PROCESS DETAILED VIEW
=====
Time Quanta: Q0=2, Q1=4, Q2=8, Q3=16 ticks
Boost Interval: Every 50 ticks all processes -> Q0
Key Test: Q3 stays at Q3 even after 16+ slices until BOOST

=====
TICK | QUEUE | SLICES | EVENT
=====
32 | Q0 | 0 | *** BOOST #1 - START TRACKING ***
33 | Q0 | 1 | Running at Q0 (slices: 1)
34 | Q1 | 0 | DEMOTE Q0->Q1 (used 2 ticks)
35 | Q1 | 1 | Running at Q1 (slices: 1)
36 | Q1 | 2 | Running at Q1 (slices: 2)
37 | Q1 | 3 | Running at Q1 (slices: 3)
38 | Q2 | 0 | DEMOTE Q1->Q2 (used 4 ticks)
39 | Q2 | 1 | Running at Q2 (slices: 1)
40 | Q2 | 2 | Running at Q2 (slices: 2)
41 | Q2 | 3 | Running at Q2 (slices: 3)
42 | Q2 | 4 | Running at Q2 (slices: 4)
43 | Q2 | 5 | Running at Q2 (slices: 5)
44 | Q2 | 6 | Running at Q2 (slices: 6)
45 | Q2 | 7 | Running at Q2 (slices: 7)
46 | Q3 | 0 | DEMOTE Q2->Q3 (used 8 ticks)

>>> Now at LOWEST priority (Q3) - will stay here until BOOST <<<
48 | Q3 | 2 | Waiting at Q3 (slices: 2/16)
49 | Q3 | 4 | Waiting at Q3 (slices: 4/16)
50 | Q3 | 6 | Waiting at Q3 (slices: 6/16)
51 | Q3 | 8 | Waiting at Q3 (slices: 8/16)
52 | Q3 | 10 | Waiting at Q3 (slices: 10/16)
53 | Q3 | 12 | Waiting at Q3 (slices: 12/16)
54 | Q3 | 14 | Waiting at Q3 (slices: 14/16)
55 | Q3 | 0 | Waiting at Q3 (slices: 0/16)
56 | Q3 | 2 | Waiting at Q3 (slices: 2/16)
57 | Q3 | 4 | Waiting at Q3 (slices: 4/16)
58 | Q3 | 6 | Waiting at Q3 (slices: 6/16)
59 | Q3 | 8 | Waiting at Q3 (slices: 8/16)
60 | Q3 | 10 | Waiting at Q3 (slices: 10/16)
61 | Q3 | 12 | Waiting at Q3 (slices: 12/16)
62 | Q3 | 14 | Waiting at Q3 (slices: 14/16)
63 | Q3 | 0 | Waiting at Q3 (slices: 0/16)
64 | Q3 | 2 | Waiting at Q3 (slices: 2/16)
65 | Q3 | 4 | Waiting at Q3 (slices: 4/16)
66 | Q3 | 6 | Waiting at Q3 (slices: 6/16)
67 | Q3 | 8 | Waiting at Q3 (slices: 8/16)
68 | Q3 | 10 | Waiting at Q3 (slices: 10/16)
69 | Q3 | 12 | Waiting at Q3 (slices: 12/16)
70 | Q3 | 14 | Waiting at Q3 (slices: 14/16)
71 | Q3 | 0 | Waiting at Q3 (slices: 0/16)
72 | Q3 | 2 | Waiting at Q3 (slices: 2/16)
73 | Q3 | 4 | Waiting at Q3 (slices: 4/16)
74 | Q3 | 6 | Waiting at Q3 (slices: 6/16)
75 | Q3 | 8 | Waiting at Q3 (slices: 8/16)
76 | Q3 | 10 | Waiting at Q3 (slices: 10/16)
77 | Q3 | 12 | Waiting at Q3 (slices: 12/16)
78 | Q3 | 14 | Waiting at Q3 (slices: 14/16)
79 | Q3 | 0 | Waiting at Q3 (slices: 0/16)
80 | Q3 | 2 | Waiting at Q3 (slices: 2/16)

=====
82 | Q0 | 0 | *** BOOST #2! Q3->Q0 ***
83 | Q0 | 1 | Running at Q0 (slices: 1)
84 | Q1 | 0 | DEMOTE Q0->Q1 (used 2 ticks)
85 | Q1 | 1 | Running at Q1 (slices: 1)
86 | Q1 | 2 | Running at Q1 (slices: 2)
87 | Q1 | 3 | Running at Q1 (slices: 3)
88 | Q2 | 0 | DEMOTE Q1->Q2 (used 4 ticks)
89 | Q2 | 1 | Running at Q2 (slices: 1)
90 | Q2 | 2 | Running at Q2 (slices: 2)

```

Figure 1: Priority boost test - demotions and first boost cycles


```

88 | Q3 | 2 | Waiting at Q3 (slices: 2/16)
-----
82 | Q0 | 0 | *** BOOST #2! Q3->Q0 ***
-----
83 | Q0 | 1 | Running at Q0 (slices: 1)
84 | Q1 | 0 | DEMOTE Q0->Q1 (used 2 ticks)
85 | Q1 | 1 | Running at Q1 (slices: 1)
86 | Q1 | 2 | Running at Q1 (slices: 2)
87 | Q1 | 3 | Running at Q1 (slices: 3)
88 | Q2 | 0 | DEMOTE Q1->Q2 (used 4 ticks)
89 | Q2 | 1 | Running at Q2 (slices: 1)
90 | Q2 | 2 | Running at Q2 (slices: 2)
91 | Q2 | 3 | Running at Q2 (slices: 3)
92 | Q2 | 4 | Running at Q2 (slices: 4)
93 | Q2 | 5 | Running at Q2 (slices: 5)
94 | Q2 | 6 | Running at Q2 (slices: 6)
95 | Q2 | 7 | Running at Q2 (slices: 7)
96 | Q3 | 0 | DEMOTE Q2->Q3 (used 8 ticks)
-----
>>> Now at LOWEST priority (Q3) - will stay here until BOOST <<<
-----
98 | Q3 | 2 | Waiting at Q3 (slices: 2/16)
100 | Q3 | 4 | Waiting at Q3 (slices: 4/16)
102 | Q3 | 6 | Waiting at Q3 (slices: 6/16)
104 | Q3 | 8 | Waiting at Q3 (slices: 8/16)
106 | Q3 | 10 | Waiting at Q3 (slices: 10/16)
108 | Q3 | 12 | Waiting at Q3 (slices: 12/16)
110 | Q3 | 14 | Waiting at Q3 (slices: 14/16)
112 | Q3 | 0 | Waiting at Q3 (slices: 0/16)
114 | Q3 | 2 | Waiting at Q3 (slices: 2/16)
116 | Q3 | 4 | Waiting at Q3 (slices: 4/16)
118 | Q3 | 6 | Waiting at Q3 (slices: 6/16)
120 | Q3 | 8 | Waiting at Q3 (slices: 8/16)
122 | Q3 | 10 | Waiting at Q3 (slices: 10/16)
124 | Q3 | 12 | Waiting at Q3 (slices: 12/16)
126 | Q3 | 14 | Waiting at Q3 (slices: 14/16)
128 | Q3 | 0 | Waiting at Q3 (slices: 0/16)
130 | Q3 | 2 | Waiting at Q3 (slices: 2/16)
-----
132 | Q0 | 0 | *** BOOST #3! Q3->Q0 ***
-----

=====
TEST SUMMARY
=====
Total Runtime : 132 ticks (~13.2 seconds)
Boost Events : 3 automatic boosts detected
Expected Interval: ~50 ticks between boosts

SUCCESS: Priority boosting is working correctly!

The test demonstrated:
1. Demotion: Q0 -> Q1 -> Q2 -> Q3 (CPU-bound behavior)
2. Staying at Q3 until boost interval
3. Q3 STAYS Q3 even after 16+ slices (no further demotion)
4. Automatic boost back to Q0 (starvation prevention)
5. Multiple boost cycles confirming periodic boosting
=====
$

```

Figure 2: Priority boost test - continued boost cycles and completion

6.2 Priority Boost Test (Short)

A quick demonstration of the priority boosting mechanism using the `boost_t` test program.

```

$ boost.t
MLFQ BOOST TEST -- Demotion & Promotion Cycles

TEST 1: Initial Priority Check
[Tick 0] Initial: PID=19, Queue=Q0, Slices=1/2
✓ PASS: New process starts at highest priority (Q0)

TEST 2: Demotion Test (CPU-bound work)
Running CPU-intensive work to trigger demotion...

Queue | Time Quantum | Meaning
-----|-----|-----
Q0     | 2 ticks       | Highest priority, shortest
Q1     | 4 ticks       |
Q2     | 8 ticks       |
Q3     | 16 ticks      | Lowest priority, longest

Note: 1 tick = 100ms. Process demotes when slices >= quantum.
Monitoring slices and demotions:

TICK 2: Q0 slices 0->1 (need 2 for demotion)
> TICK 3: DEMOTED Q0->Q1 (used 2/2 slices)
TICK 4: Q1 slices 0->1 (need 4 for demotion)
TICK 5: Q1 slices 1->2 (need 4 for demotion)
TICK 6: Q1 slices 2->3 (need 4 for demotion)
> TICK 7: DEMOTED Q1->Q2 (used 4/4 slices)
TICK 8: Q2 slices 0->1 (need 8 for demotion)
TICK 9: Q2 slices 1->2 (need 8 for demotion)
TICK 10: Q2 slices 2->3 (need 8 for demotion)

[Tick 11] After work: PID=19, Queue=Q2, Slices=3/8

Demotion Summary:
Q0->Q1 at tick 3 (expected after 2 ticks in Q0)
Q1->Q2 at tick 7 (expected after 4 more ticks in Q1)
✓ PASS: Process demoted from Q0 to Q2

TEST 3: Manual Boost (boostproc syscall)
[Tick 11] Before boost: PID=19, Queue=Q2, Slices=3/8
Calling boostproc()...
[Tick 11] After boost: PID=19, Queue=Q0, Slices=0/2
Boost executed between tick 11 and 11
✓ PASS: Process boosted to Q0 with slices reset

TEST 4: Re-demotion After Boost
Verifying demotion still works after manual boost...
(Note: Auto-boost may interfere if BOOST_INTERVAL is short)
> TICK 14: DEMOTED Q0->Q1
[Tick 14] Final: PID=19, Queue=Q1, Slices=0/4
✓ PASS: Demotion mechanism working after boost

TEST 5: Automatic Periodic Boost
Waiting and working until automatic boost triggers...
This tests starvation prevention mechanism.
> TICK 18: Demotion Q1->Q2
> TICK 26: Demotion Q2->Q3
< TICK 31: AUTO BOOST DETECTED Q3->Q0
✓ PASS: Automatic periodic boost working!

TEST SUMMARY
Total runtime: 31 ticks (~3.1 seconds)
Initial priority: Q0
Max demotion reached: Q2
Manual boost working
Auto boost: Detected

✓✓✓ ALL CORE TESTS PASSED ✓✓✓
$

```

Figure 3: Priority boost demonstration

6.3 Starvation Prevention Test

The `starve.t` test verifies that lower-priority processes do not starve due to periodic priority boosts. Even when high-priority processes are running, the boost mechanism ensures all processes get CPU time.

```

$ starve.t
== Starvation Prevention Test (Week 3) ==
Testing automatic priority boosting every 100 ticks
Running 3 CPU-bound processes concurrently
All processes should get CPU time due to periodic boosting

[Process 0] Started (PID=16)
[Process 0] Tick 2787: Q0 (slices=0)
[Process 1] Started (PID=17)
[Process 1] Tick 2789: Q0 (slices=0)

[Parent] Waiting for all processes to complete...
[Process 2] Started (PID=18)
[Process 2] Tick 2791: Q0 (slices=0)
[Process 0] Tick 2796: Q1 (slices=3)
[Process 1] Tick 2802: Q0 (slices=0)
[Process 0] Tick 2808: Q1 (slices=2)
[Process 1] Tick 2813: Q1 (slices=3)
[Process 2] Tick 2815: Q1 (slices=1)
[Process 0] Tick 2821: Q2 (slices=3)
[Process 0] COMPLETED at tick 2825: Final Q2
[Parent] Process 0 finished
[Process 1] Tick 2829: Q2 (slices=4)
[Process 2] Tick 2835: Q2 (slices=2)
[Process 2] Tick 2839: Q2 (slices=6)
[Process 1] COMPLETED at tick 2843: Final Q3
[Parent] Process 1 finished
[Process 2] COMPLETED at tick 2846: Final Q3
[Parent] Process 2 finished

== Starvation Test Complete ==
Analysis:
- All processes should have completed
- Processes should have been boosted to Q0 every ~100 ticks
- Even low-priority processes got CPU time
- No process was starved!
$

```

Figure 4: Starvation prevention through periodic priority boosting

6.4 Priority Boost Test (Long)

The `boostlong` test demonstrates automatic priority boosting every 50 ticks, showing the complete demotion cycle ($Q0 \rightarrow Q1 \rightarrow Q2 \rightarrow Q3$) and how processes at $Q3$ stay at $Q3$ until boosted back to $Q0$.

```
$
$ boostlong

=====
MLFQ BOOST TEST - SINGLE PROCESS DETAILED VIEW
=====
Time Quanta: Q0=2, Q1=4, Q2=8, Q3=16 ticks
Boost Interval: Every 50 ticks all processes -> Q0
Key Test: Q3 stays at Q3 even after 16+ slices until BOOST

=====
TICK | QUEUE | SLICES | EVENT
=====
32 | Q0 | 0 | *** BOOST #1 - START TRACKING ***
33 | Q0 | 1 | Running at Q0 (slices: 1)
34 | Q1 | 0 | DEMOTE Q0->Q1 (used 2 ticks)
35 | Q1 | 1 | Running at Q1 (slices: 1)
36 | Q1 | 2 | Running at Q1 (slices: 2)
37 | Q1 | 3 | Running at Q1 (slices: 3)
38 | Q2 | 0 | DEMOTE Q1->Q2 (used 4 ticks)
39 | Q2 | 1 | Running at Q2 (slices: 1)
40 | Q2 | 2 | Running at Q2 (slices: 2)
41 | Q2 | 3 | Running at Q2 (slices: 3)
42 | Q2 | 4 | Running at Q2 (slices: 4)
43 | Q2 | 5 | Running at Q2 (slices: 5)
44 | Q2 | 6 | Running at Q2 (slices: 6)
45 | Q2 | 7 | Running at Q2 (slices: 7)
46 | Q3 | 0 | DEMOTE Q2->Q3 (used 8 ticks)

>>> Now at LOWEST priority (Q3) - will stay here until BOOST <<<
48 | Q3 | 2 | Waiting at Q3 (slices: 2/16)
50 | Q3 | 4 | Waiting at Q3 (slices: 4/16)
52 | Q3 | 6 | Waiting at Q3 (slices: 6/16)
54 | Q3 | 8 | Waiting at Q3 (slices: 8/16)
56 | Q3 | 10 | Waiting at Q3 (slices: 10/16)
58 | Q3 | 12 | Waiting at Q3 (slices: 12/16)
60 | Q3 | 14 | Waiting at Q3 (slices: 14/16)
62 | Q3 | 0 | Waiting at Q3 (slices: 0/16)
64 | Q3 | 2 | Waiting at Q3 (slices: 2/16)
66 | Q3 | 4 | Waiting at Q3 (slices: 4/16)
68 | Q3 | 6 | Waiting at Q3 (slices: 6/16)
70 | Q3 | 8 | Waiting at Q3 (slices: 8/16)
72 | Q3 | 10 | Waiting at Q3 (slices: 10/16)
74 | Q3 | 12 | Waiting at Q3 (slices: 12/16)
76 | Q3 | 14 | Waiting at Q3 (slices: 14/16)
78 | Q3 | 0 | Waiting at Q3 (slices: 0/16)
80 | Q3 | 2 | Waiting at Q3 (slices: 2/16)

=====
82 | Q0 | 0 | *** BOOST #2! Q3->Q0 ***
83 | Q0 | 1 | Running at Q0 (slices: 1)
84 | Q1 | 0 | DEMOTE Q0->Q1 (used 2 ticks)
85 | Q1 | 1 | Running at Q1 (slices: 1)
86 | Q1 | 2 | Running at Q1 (slices: 2)
87 | Q1 | 3 | Running at Q1 (slices: 3)
88 | Q2 | 0 | DEMOTE Q1->Q2 (used 4 ticks)
89 | Q2 | 1 | Running at Q2 (slices: 1)
90 | Q2 | 2 | Running at Q2 (slices: 2)
```

Figure 5: Priority boost test - demotions and first boost cycles

```

80 | Q3 | 2 | Waiting at Q3 (slices: 2/16)
-----
82 | Q0 | 0 | *** BOOST #2! Q3->Q0 ***
-----
83 | Q0 | 1 | Running at Q0 (slices: 1)
84 | Q1 | 0 | DEMOTE Q0->Q1 (used 2 ticks)
85 | Q1 | 1 | Running at Q1 (slices: 1)
86 | Q1 | 2 | Running at Q1 (slices: 2)
87 | Q1 | 3 | Running at Q1 (slices: 3)
88 | Q2 | 0 | DEMOTE Q1->Q2 (used 4 ticks)
89 | Q2 | 1 | Running at Q2 (slices: 1)
90 | Q2 | 2 | Running at Q2 (slices: 2)
91 | Q2 | 3 | Running at Q2 (slices: 3)
92 | Q2 | 4 | Running at Q2 (slices: 4)
93 | Q2 | 5 | Running at Q2 (slices: 5)
94 | Q2 | 6 | Running at Q2 (slices: 6)
95 | Q2 | 7 | Running at Q2 (slices: 7)
96 | Q3 | 0 | DEMOTE Q2->Q3 (used 8 ticks)
-----
>>> Now at LOWEST priority (Q3) -- will stay here until BOOST <<<
-----
98 | Q3 | 2 | Waiting at Q3 (slices: 2/16)
100 | Q3 | 4 | Waiting at Q3 (slices: 4/16)
102 | Q3 | 6 | Waiting at Q3 (slices: 6/16)
104 | Q3 | 8 | Waiting at Q3 (slices: 8/16)
106 | Q3 | 10 | Waiting at Q3 (slices: 10/16)
108 | Q3 | 12 | Waiting at Q3 (slices: 12/16)
110 | Q3 | 14 | Waiting at Q3 (slices: 14/16)
112 | Q3 | 0 | Waiting at Q3 (slices: 0/16)
114 | Q3 | 2 | Waiting at Q3 (slices: 2/16)
116 | Q3 | 4 | Waiting at Q3 (slices: 4/16)
118 | Q3 | 6 | Waiting at Q3 (slices: 6/16)
120 | Q3 | 8 | Waiting at Q3 (slices: 8/16)
122 | Q3 | 10 | Waiting at Q3 (slices: 10/16)
124 | Q3 | 12 | Waiting at Q3 (slices: 12/16)
126 | Q3 | 14 | Waiting at Q3 (slices: 14/16)
128 | Q3 | 0 | Waiting at Q3 (slices: 0/16)
130 | Q3 | 2 | Waiting at Q3 (slices: 2/16)
-----
132 | Q0 | 0 | *** BOOST #3! Q3->Q0 ***
-----

=====
TEST SUMMARY
=====
Total Runtime : 132 ticks (~13.2 seconds)
Boost Events : 3 automatic boosts detected
Expected Interval: ~50 ticks between boosts

SUCCESS: Priority boosting is working correctly!

The test demonstrated:
1. Demotion: Q0 -> Q1 -> Q2 -> Q3 (CPU-bound behavior)
2. Staying at Q3 until boost interval
3. Q3 STAYS Q3 even after 16+ slices (no further demotion)
4. Automatic boost back to Q0 (starvation prevention)
5. Multiple boost cycles confirming periodic boosting
=====
$

```

Figure 6: Priority boost test - continued boost cycles and completion

6.5 Starvation Prevention Test

The `starve_t` test verifies that lower-priority processes do not starve due to periodic priority boosts. Even when high-priority processes are running, the boost mechanism ensures all processes get CPU time.

```

$ starve_t
=== Starvation Prevention Test (Week 3) ===
Testing automatic priority boosting every 100 ticks
Running 3 CPU-bound processes concurrently
All processes should get CPU time due to periodic boosting

[Process 0] Started (PID=16)
[Process 0] Tick 2787: Q0 (slices=0)
[Process 1] Started (PID=17)
[Process 1] Tick 2789: Q0 (slices=0)

[Parent] Waiting for all processes to complete...
[Process 2] Started (PID=18)
[Process 2] Tick 2791: Q0 (slices=0)
[Process 0] Tick 2796: Q1 (slices=3)
[Process 1] Tick 2802: Q0 (slices=0)
[Process 0] Tick 2808: Q1 (slices=2)
[Process 1] Tick 2813: Q1 (slices=3)
[Process 2] Tick 2815: Q1 (slices=1)
[Process 0] Tick 2821: Q2 (slices=3)
[Process 0] COMPLETED at tick 2825: Final Q2
[Parent] Process 0 finished
[Process 1] Tick 2829: Q2 (slices=4)
[Process 2] Tick 2836: Q2 (slices=2)
[Process 2] Tick 2839: Q2 (slices=6)
[Process 1] COMPLETED at tick 2843: Final Q3
[Parent] Process 1 finished
[Process 2] COMPLETED at tick 2846: Final Q3
[Parent] Process 2 finished

=== Starvation Test Complete ===
Analysis:
- All processes should have completed
- Processes should have been boosted to Q0 every ~100 ticks
- Even low-priority processes got CPU time
- No process was starved!
$

```

Figure 7: Starvation prevention through periodic priority boosting

Test	Expected Behavior	Result
procinfo	Returns correct process info	PASS
mlfqtest	I/O gets priority over CPU	PASS
cpubound	Demotes Q0 $\rightarrow$ Q3	PASS
iobound	Stays in Q0	PASS
fairness_t	Fair CPU distribution	PASS
boost_t	Shows priority boost	PASS
boostlong	Shows 3+ boost cycles	PASS
starve_t	No process starvation	PASS

Table 4: Final Test Results

6.6 Test Summary

6.7 Key Observations

1. **Demotion Works:** CPU-bound processes correctly demote through all queues (Q0 $\rightarrow$ Q1 $\rightarrow$ Q2 $\rightarrow$ Q3)
2. **I/O Priority:** Processes that yield frequently maintain high priority (stay in Q0/Q1)
3. **Boosting Works:** All processes are boosted every 50 ticks back to Q0
4. **Q3 Behavior:** Processes at Q3 stay at Q3 (slices reset to 0 but queue unchanged) until boost
5. **Starvation Prevented:** No process is starved indefinitely due to periodic boosting
6. **Fairness:** Mixed workloads demonstrate correct priority differentiation

7 Performance Characteristics

7.1 Responsiveness

- Interactive processes stay in Q0/Q1 with 2-4 tick quanta
- Response time: 200-400ms for I/O-bound processes
- System feels responsive even under CPU load

7.2 Throughput

- CPU-bound processes get longer quanta at Q3 (16 ticks)
- Reduced context switching for compute-intensive work
- Good overall CPU utilization

7.3 Fairness

- Boosting every 50 ticks ensures no starvation
- All processes get periodic high-priority access
- Round-robin within each queue ensures fairness among peers

8 Conclusion

8.1 Achievements

Successfully implemented 4-level MLFQ scheduler

Automatic demotion based on CPU usage

Priority boosting prevents starvation (every 50 ticks)

I/O-bound processes receive preferential treatment

Comprehensive test suite validates all features

System remains stable and responsive

8.2 MLFQ Rules Implemented

1. **Rule 1:** If $\text{Priority}(A) > \text{Priority}(B)$, A runs
2. **Rule 2:** If $\text{Priority}(A) = \text{Priority}(B)$, round-robin
3. **Rule 3:** New processes start at highest priority (Q0)
4. **Rule 4:** If process uses full quantum, demote
5. **Rule 5:** After boost interval (50 ticks), all processes $\rightarrow$ Q0

8.3 Future Improvements

Potential enhancements for future work:

- Gaming prevention (detect processes that reset timers)
- Configurable queue parameters
- Per-process priority adjustment syscall
- Multi-core MLFQ scheduling
- Real-time priority class support

9 Appendix: System Call Reference

9.1 getprocinfo

```
1 int getprocinfo(struct procinfo *info);
```

Description: Retrieves scheduling information for the calling process.

Returns: 0 on success, -1 on failure.

Structure:

```
1 struct procinfo {  
2     int pid;           // Process ID  
3     int state;         // Process state  
4     int priority;      // Current queue (0-3)  
5     int time_slices;   // Slices used in current queue  
6 };
```

9.2 boostproc

```
1 int boostproc(void);
```

Description: Manually triggers priority boosting for all processes.

Returns: 0 on success.

Use Case: Testing and debugging priority boost behavior.